

CSA-0619 CO1 AT1 ANSWERS

Q22. Application: Digital Library Search Engine A digital library system processes search queries using the recurrence $T(n)=T(n-1)+\log_{10} n$. Apply the substitution method to solve the recurrence. Expand step-by-step and derive the final complexity. Analyze how performance changes with increasing documents. Interpret the result using asymptotic notation

Q22. Digital Library Search Engine — Substitution Method Analysis

1. Recurrence Relation

$$T(n) = T(n-1) + \log n$$

2. Substitution Method — Step-by-Step Expansion

Start with the recurrence:

$$T(n) = T(n-1) + \log n$$

Step 1: Expand $T(n-1)$

$$\begin{aligned} T(n-1) &= T(n-2) + \log(n-1) \\ \Rightarrow T(n) &= T(n-2) + \log(n-1) + \log n \end{aligned}$$

Step 2: Expand $T(n-2)$

$$\begin{aligned} T(n-2) &= T(n-3) + \log(n-2) \\ \Rightarrow T(n) &= T(n-3) + \log(n-2) + \log(n-1) + \log n \end{aligned}$$

Step 3: Continue expanding until we reach the base case $T(1)$ (or $T(0)$)

After k expansions:

$$T(n) = T(n-k) + \log(n-k+1) + \log(n-k+2) + \dots + \log n$$

Step 4: Stop when $n - k = 1$, i.e., $k = n - 1$

$$T(n) = T(1) + \log(2) + \log(3) + \dots + \log(n)$$

3. Summation Form

$$T(n) = T(1) + \sum_{i=2}^n \log i$$

This is equivalent to:

$$T(n) = T(1) + \log(2 \times 3 \times 4 \times \dots \times n)$$

$$T(n) = T(1) + \log(n!)$$

4. Solving the Summation

Using the known bound (from Stirling's approximation):

$$\log(n!) = \Theta(n \log n)$$

Therefore:

$$T(n) = T(1) + \Theta(n \log n)$$

$$T(n) = \Theta(n \log n)$$

5. Verification by Substitution (Guess and Prove)

Guess: $T(n) = O(n \log n)$

Assume: $T(k) \leq c \cdot k \cdot \log k$ for all $k < n$

Prove for n:

$$\begin{aligned} T(n) &= T(n-1) + \log n \\ &\leq c(n-1)\log(n-1) + \log n \\ &\leq c \cdot n \cdot \log n \quad (\text{for sufficiently large } c \text{ and } n) \end{aligned}$$

Hence, by induction, $T(n) = O(n \log n)$ — verified.

Similarly, a lower bound argument gives $T(n) = \Omega(n \log n)$, confirming:

$$T(n) = \Theta(n \log n)$$

6. Final Time Complexity

$$T(n) = \Theta(n \log n)$$

7. Performance Analysis for Increasing Documents (Large n)

- The recurrence models a digital library search where processing **n documents** requires solving a smaller subproblem of **$(n-1)$ documents**, plus an additional **$\log n$** cost at each step — representing operations like searching an index, comparing metadata, or narrowing a sorted document range (e.g., binary-search-like indexing cost).
- Since the recursive depth is **linear ($n-1$ calls)** and each call contributes a **logarithmic** cost, the cumulative cost sums to **$n \log n$** .
- As the number of documents **n** grows (say from thousands to millions), the search engine's total processing cost grows **slightly faster than linear time**, but remains far more efficient than **quadratic (n^2)** growth.
- This makes the algorithm **scalable** for large digital libraries — a good balance between simplicity (linear recursive structure) and reasonably fast per-step operations (logarithmic).

8. Interpretation Using Asymptotic Notation

Upper Bound (Big-O): $T(n) = O(n \log n)$

Lower Bound (Big- Ω): $T(n) = \Omega(n \log n)$

Tight Bound (Big- Θ): $T(n) = \Theta(n \log n)$

Interpretation: The digital library search engine exhibits **$\Theta(n \log n)$** time complexity. This indicates that as the document collection grows, the total query-processing cost increases slightly faster than linearly, but remains efficient and scalable — comparable to the performance of well-optimized sorting or search-indexing algorithms. This complexity is considered **efficient and practical** for real-world large-scale digital libraries.